CHAPTER 9

Testing JavaScript: Integration Testing

Nearly every Rails site uses JavaScript for something, and often JavaScript is a critical part of the user experience on sites. However, many sites don't test their JavaScript in any meaningful way.

Part of the reason is that JavaScript can be difficult to test. Interactions and logic are often dependent on user interaction or the specific state of the elements on the page, which can be hard to set up for testing. Often the JavaScript frameworks don't support easy unit testing. And the tools are changing very quickly. The tools I use in this chapter may seem quaint if not downright antiquated by the time you read this.

That said, some general principles for testing JavaScript will continue to be applicable across tools.

I'll talk about JavaScript testing in two parts. In this chapter I'll talk about using integration-testing tools, specifically Capybara, to test JavaScript from outside the client-side code. In the next chapter I'll talk about unit-testing JavaScript using JavaScript tools and the Webpack support that was new in Rails 5.1.

Integration-Testing JavaScript with Capybara

In the last chapter you used Capybara to test buttons in the UI that allowed you to change the order of tasks. As written, clicking up on a task button causes a full server page refresh. This is an interaction that might be more user-friendly if handled client-side with JavaScript.

There are multiple ways to handle this client-side. For our purposes the simplest is to have the up and down buttons manipulate the browser DOM to change the order and send an update request back to the server to register the other change. You need to test two things: that the client interaction